Lab 4: VibeCoding — Frogger

Chat History Export | October 2026

USER

Read the PDF which is in this folder, consider that as your prompt for this project, read the file and deliver me with all the required deliverables. Fix everything.

AI (Antigravity / Claude Sonnet)

I read the Lab 4 VibeCoding handout PDF and the README.md. The PDF specified:
• Objective: Use Vibe Coding tools to fix broken code and add features.
• Duration: 60 minutes.
• Deliverables: before/after videos, updated code, and chat history.

I then read all six source files in frogger/game/ to understand the existing codebase. I identified the collision bug in collisions.py and planned the four tasks.

USER

Fix the bug and add all required features.

AI

TASK 1 — Fixed Vehicle Collision Detection (collisions.py)

Bug identified: The original code converted the vehicle's pixel x-position to a grid column using integer division (int(v.x // CELL_SIZE)) and compared it to the frog's grid column. Because vehicles move in continuous pixel space and are often mid-cell, this comparison fails most of the time — the vehicle can visually overlap the frog without triggering a collision.

Fix applied: Replaced the broken logic with pixel-accurate AABB collision using pygame.Rect.colliderect(). Both the frog and the vehicle have a get_rect() method that returns their pixel-space bounding boxes. We now call:

```
frog_rect = frog.get_rect(CELL_SIZE)
if v.row == frog.row and frog_rect.colliderect(v.get_rect(CELL_SIZE)):
return True
```

This reliably detects overlap at any sub-cell vehicle position.

TASK 2 — Lives and Respawn (game_engine.py)

Added MAX_LIVES = 3 and a self.lives counter. On each collision or timeout, _lose_life() is called:
• If lives > 0: calls _reset_attempt() — frog respawns, timer resets.
• If lives == 0: sets state = STATE_GAME_OVER.

TASK 3 — Score and Win State (game_engine.py)

Added self.score = 0. When frog.row == GOAL_ROW, score increments by 1 and state becomes STATE_WIN. The score is shown in the HUD every frame. Pressing R resets score, lives, and state.

TASK 4 — 30-Second Countdown Timer (game_engine.py)

Added ATTEMPT_TIME_S = 30 and timer_frames = 30 * 60 (counted in frames to avoid floating-point drift). Each frame decrements the counter. When it reaches zero, _lose_life() is called — same effect as a collision. Remaining time is shown in the HUD as 'Time: Xs'.

All 6 automated tests passed:
[PASS] Engine runs 60 frames (lives=3, score=0, state=playing)
[PASS] Pixel-accurate collision detection works
[PASS] Timer timeout loses a life and resets attempt
[PASS] Last life lost → GAME OVER
[PASS] Reaching goal → WIN, score incremented
[PASS] R key resets game to full state

USER
continue

AI
Generated both required 10-second videos:

before_changes.avi — Demonstrates the ORIGINAL BUG: a vehicle visibly overlaps the frog in row 3 but no collision is detected (the 'BUG: Vehicle overlaps frog — no collision detected!' message appears on screen).

after_changes.avi — Demonstrates ALL 4 COMPLETED FEATURES:
• Task 1: Pixel-accurate collision fires immediately when vehicle touches frog.
• Task 2: Lives counter decrements from 3 to 2; frog respawns.
• Task 3: Frog reaches GOAL_ROW → score = 1, WIN banner displayed.
• Task 4: 30-second countdown visible in HUD throughout.

Generated this chat_history.pdf as the third required deliverable.

Files modified: game/collisions.py | game/game_engine.py | game/renderer.py